

Informe de normalización del modelo relacional

Plataforma de gestión de gimnasios

Octubre de 2026

1. Introducción

En este informe describimos cómo aplicamos los pasos de normalización al modelo relacional de la plataforma de gimnasios. Partimos del diagrama Entidad-Relación (E-R) de la primera entrega y llegamos a un modelo de 28 tablas, que se encuentran en el archivo Modelo_Relacional.xlsx junto con el tipo de dato de cada columna.

Para cada forma normal explicamos la regla, cómo la revisamos en nuestro modelo y qué cambios hicimos. Al final explicamos por qué decidimos no llevar el modelo a la sexta forma normal.

2. Paso del modelo E-R al modelo relacional

Antes de normalizar, transformamos el diagrama E-R en tablas con las siguientes reglas:

- **Entidades:** convertimos cada entidad en una tabla con su llave primaria (PK).
- **Relaciones 1:N:** ubicamos la llave foránea (FK) en la tabla del lado con cardinalidad 1:1. Por ejemplo, Sede recibe Nit_Empresa y Sesión recibe Cod_Servicio.
- **Relaciones 1:1:** en Cuenta empleado y Cuenta cliente ubicamos la FK en el lado de participación total; por eso Empleado y Cliente reciben Cod_Usuario.
- **Relaciones N:M:** por cada una creamos una tabla intermedia cuya PK está formada por las FK de las dos entidades: Sede_Plan, Servicio_Plan, Servicio_Sede, Sesión_Cliente y Rol_Permission. Los atributos propios de la relación los dejamos en esa tabla (por ejemplo, Cupo_Mensual y Costo_Adicional en Servicio_Plan).
- **Entidades débiles:** a Acceso, Mantenimiento y Bitacora les asignamos una PK compuesta por la llave de la entidad de la que dependen y su atributo discriminante.
- **Especialización:** con los subtipos de Empleado creamos las tablas Entrenador y Administrativo, cuya PK es a la vez FK hacia Empleado (Documento_Empleado).
- **Atributos de relaciones 1:N:** los atributos de la relación Posee (Fecha_Asignacion_Rol y Rol_Vigente) los ubicamos en Usuario, que es la tabla que recibe la FK de esa relación.

También unificamos la forma de nombrar las columnas: sin tildes ni ñ, con palabras separadas por guion bajo. A los atributos que se repetían en varias tablas (Nombre, Estado, Descripción, Teléfono, Documento, entre otros) les agregamos el nombre de su entidad, por ejemplo Nombre_Sede, Estado_Pago o Documento_Cliente. Cada llave foránea la nombramos igual que la llave primaria a la que apunta. La única excepción es Documento_Entrenador en Sesión, que apunta a la PK de Entrenador; la nombramos así para dejar claro que solo puede ser un entrenador.

3. Primera forma normal (1FN)

Regla. Cada columna debe guardar un solo valor (valores atómicos), no puede haber grupos repetidos y cada tabla debe tener una llave primaria.

Qué revisamos y qué cambiamos.

- Separamos el atributo multivaluado Teléfono de Cliente en la tabla **Telefonos_Cliente** (Telefono_Cliente, Documento_Cliente).
- Separamos el atributo multivaluado Restricciones en la tabla **Restricciones_Cliente** (Restriccion, Documento_Cliente).
- El nombre del cliente es un atributo compuesto por Nombres y Apellidos. Como un cliente puede tener varios nombres y dos apellidos, tratamos cada parte como multivaluada y la separamos en **Nombres_Cliente** y **Apellidos_Cliente**.
- Verificamos que todas las tablas tengan una llave primaria definida y que cada columna tenga un único tipo de dato.

Así quedó el cambio con los teléfonos:

Antes (no cumple 1FN)	Después (cumple 1FN)
Cliente: Documento_Cliente = 1020304050, Telefono = "3001234567, 6015551234"	Telefonos_Cliente: (3001234567, 1020304050) y (6015551234, 1020304050), una fila por teléfono

Resultado: nuestro modelo cumple 1FN.

4. Segunda forma normal (2FN)

Regla. Además de cumplir 1FN, cada atributo que no es llave debe depender de toda la llave primaria y no solo de una parte. Esta regla solo puede fallar en las tablas con llave compuesta, así que revisamos todas:

Tabla	Llave primaria	Atributos no llave	¿Dependen de toda la llave?
Acceso	Cod_Membresia + Fecha_Hora_Entrada	Fecha_Hora_Salida, Cod_Sede	Sí: la salida y la sede corresponden a un ingreso concreto, no a la membresía en general.
Bitacora	Cod_Usuario + Fecha_Hora_Ingreso	Direccion_IP, Dispositivo, Resultado	Sí: describen un intento de inicio de sesión concreto.
Mantenimiento	Serial + N_Reporte	Fecha_Mantenimiento, Descripcion_Mantenimiento, Costo	Sí: describen un reporte de un equipo.
Servicio_Plan	Cod_Servicio + Cod_Plan	Cupo_Mensual, Costo_Adicional	Sí: cambian según la combinación de servicio y plan.
Servicio_Sede	Cod_Servicio + Cod_Sede	Horario	Sí: un mismo servicio tiene horario distinto en cada sede.
Sesion_Cliente	Cod_Sesion + Documento_Cliente	Fecha_Reserva, Estado_Reserva	Sí: pertenecen a la reserva de un cliente en una sesión.
Sede_Plan, Rol_Permiso y las 4 tablas de atributos multivaluados	Compuesta	Ninguno	Cumplen directamente.

Por ejemplo, no guardamos Nombre_Servicio en Servicio_Plan, porque dependería solo de Cod_Servicio (una parte de la llave). Ese dato lo consultamos en la tabla Servicio.

Resultado: nuestro modelo cumple 2FN.

5. Tercera forma normal (3FN)

Regla. Además de cumplir 2FN, ningún atributo que no es llave puede depender de otro atributo que tampoco es llave (dependencia transitiva).

Qué revisamos y qué decidimos.

- **Pago.** Guardamos Cod_Membresia y Documento_Empleado (el empleado que registra el pago, según la relación Registra del diagrama). No guardamos el documento del cliente, porque ya se obtiene a través de la membresía (N_Factura → Cod_Membresia → Documento_Cliente). Guardarlo en Pago habría creado una dependencia transitiva.
- **Solo llaves foráneas.** En cada tabla guardamos únicamente la FK de otras entidades, nunca sus datos descriptivos. Por ejemplo, Sesion guarda Cod_Espacio pero no Aforo_Espacio, y Membresia guarda Cod_Plan pero no Precio.
- **Plan.** Definimos Tipo_Plan como la categoría del plan (por ejemplo, Básico o Premium), así que no determina Duracion_Meses y no hay dependencia transitiva entre ellos.
- **Membresia.Fecha_Fin.** Se podría calcular con Fecha_Inicio y la duración del plan, pero decidimos conservarla porque registra la fecha pactada en el momento de la venta. Si más adelante cambia la duración del plan, las membresías ya vendidas no deben cambiar, por lo que Fecha_Fin no depende de otros atributos de la tabla.
- **Usuario.Ultimo_Ingreso.** Es un dato derivado de Bitacora que decidimos mantener porque se consulta con frecuencia. Dentro de Usuario depende solo de Cod_Usuario, así que no crea una dependencia transitiva.
- **Correo_Usuario y Correo_Cliente.** Conservamos ambos porque el primero es la credencial para iniciar sesión y el segundo es el correo de contacto del cliente, y pueden ser distintos.

Resultado: nuestro modelo cumple 3FN.

6. Forma normal de Boyce-Codd (FNBC)

Regla. Todo atributo (o grupo de atributos) que determine a otros debe ser una llave candidata.

Además de las llaves primarias, encontramos que los únicos determinantes del modelo son valores únicos, es decir, llaves candidatas:

- Correo_Usuario en Usuario: cada correo pertenece a un solo usuario.
- Cod_Usuario en Empleado y en Cliente: por las relaciones 1:1, cada usuario corresponde a un solo empleado o cliente.
- Modulo + Accion en Permiso: cada combinación define un único permiso.

Resultado: nuestro modelo cumple FNBC.

7. Cuarta forma normal (4FN)

Regla. Una tabla no debe guardar dos hechos multivaluados que sean independientes entre sí.

Dejamos los cuatro atributos multivaluados del cliente (nombres, apellidos, teléfonos y restricciones) en tablas separadas. Si los hubiéramos juntado, por ejemplo en una tabla `Cliente_Datos(Documento_Cliente, Telefono_Cliente, Restriccion)`, habría que repetir cada teléfono por cada restricción: un cliente con 2 teléfonos y 3 restricciones ocuparía 6 filas. Al separarlos, cada dato se guarda una sola vez. Nuestras tablas de relaciones N:M son de dos entidades y no mezclan hechos independientes.

Resultado: nuestro modelo cumple 4FN.

8. Quinta forma normal (5FN)

Regla. Una tabla no se debe poder dividir en tablas más pequeñas que, al volver a unirlas, den exactamente la misma información (dependencias de reunión), salvo las que se deriven de sus llaves.

El único caso que aplica en nuestro modelo es el triángulo Plan–Sede–Servicio, formado por tres tablas:

- **Sede_Plan:** qué sedes cubre cada plan.
- **Servicio_Plan:** qué servicios incluye cada plan.
- **Servicio_Sede:** qué servicios presta cada sede.

Definimos la siguiente regla de negocio: un afiliado puede usar un servicio en una sede si su plan cubre esa sede, su plan incluye ese servicio y la sede presta ese servicio. Con esa regla, al unir las tres tablas reconstruimos la información completa sin generar combinaciones falsas, por lo que no necesitamos una tabla de tres llaves (`Plan_Servicio_Sede`).

Resultado: nuestro modelo cumple 5FN.

9. Sexta forma normal (6FN): por qué no la aplicamos

Regla. Cada tabla debe tener su llave y, como máximo, un atributo adicional.

Aplicarla nos obligaría a dividir cada tabla en una tabla por atributo. Por ejemplo, `Cliente` quedaría así:

- `Cliente_Fecha_Nacimiento(Documento_Cliente, Fecha_Nacimiento)`
- `Cliente_Correo(Documento_Cliente, Correo_Cliente)`
- `Cliente_RH(Documento_Cliente, RH)`
- `Cliente_Usuario(Documento_Cliente, Cod_Usuario)`

Si hiciéramos lo mismo en todo el modelo, pasaríamos de 28 a más de 90 tablas. Decidimos no aplicarla por estas razones:

- **Demasiados JOIN en las búsquedas.** Por ejemplo, para listar los pagos con su fecha, monto, método y el nombre del plan, hoy unimos 3 tablas (`Pago`, `Membresia` y `Plan`) con 2

JOIN. En 6FN necesitaríamos 6 tablas (Pago_Fecha, Pago_Monto, Pago_Metodo, Pago_Membresia, Membresia_Plan y Plan_Nombre) y 5 JOIN. Esto se repetiría en casi todas las consultas de la aplicación y las haría más lentas y difíciles de escribir.

- **Registros más costosos.** Para crear un solo cliente tendríamos que escribir en varias tablas a la vez dentro de la misma transacción.
- **No aporta beneficios en nuestro caso.** La 6FN está pensada para bases de datos temporales y bodegas de datos que guardan el historial de cada atributo por separado. Para nuestra aplicación, la 5FN ya elimina la redundancia y las anomalías de inserción, actualización y borrado.

10. Conclusión

Nuestro modelo relacional final tiene 28 tablas y cumple hasta la quinta forma normal. No aplicamos la sexta forma normal porque generaría demasiados JOIN en las búsquedas sin aportar beneficios para la aplicación.

Forma normal	¿Cumple?	Qué hicimos
1FN	Sí	Pasamos nombres, apellidos, teléfonos y restricciones del cliente a tablas propias.
2FN	Sí	Verificamos que los atributos de las tablas con llave compuesta dependan de toda la llave.
3FN	Sí	En Pago guardamos Documento_Empleado y no el del cliente; las tablas solo guardan FK hacia otras entidades.
FNBC	Sí	Comprobamos que los determinantes adicionales (Correo_Usuario, Cod_Usuario, Modulo + Accion) son llaves candidatas.
4FN	Sí	Dejamos los atributos multivaluados independientes en tablas separadas.
5FN	Sí	Resolvimos el triángulo Plan–Sede–Servicio con tres tablas binarias.
6FN	No (decidimos no aplicarla)	Pasaríamos a más de 90 tablas y las búsquedas necesitarían demasiados JOIN.